Section 2

By Tan Song Xin, Alastair

System Design:

Figure 1 below shows the overall system architecture.

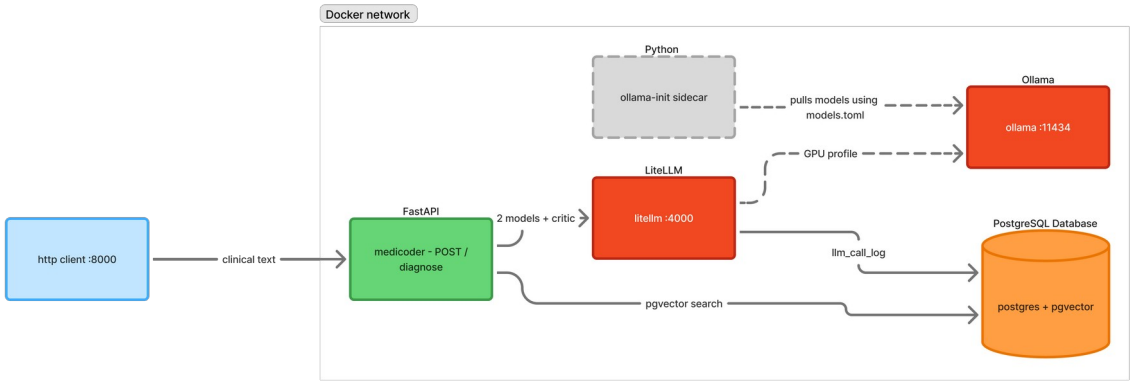


Figure 1: Overall System Architecture

5 Docker containers are started up as part of this project, although 1 is an ephermeral sidecar container called *ollama-init* that shutdowns upon completion of its task. This sidecar was used as a function caller to get the Ollama container to pull the required models to the container upon startup, saving time as it is atypical for a /diagnose API call to be made upon immediate system startup.

Containers are used to enforce the seperation of responsibility and loose coupling. Implementation changes in LiteLLM should not affect Ollama, and changes in PostgreSQL Database's stack will not cause any knock-on effects.

Figure 2 shows the diagram of all the structure of the medicoder container.

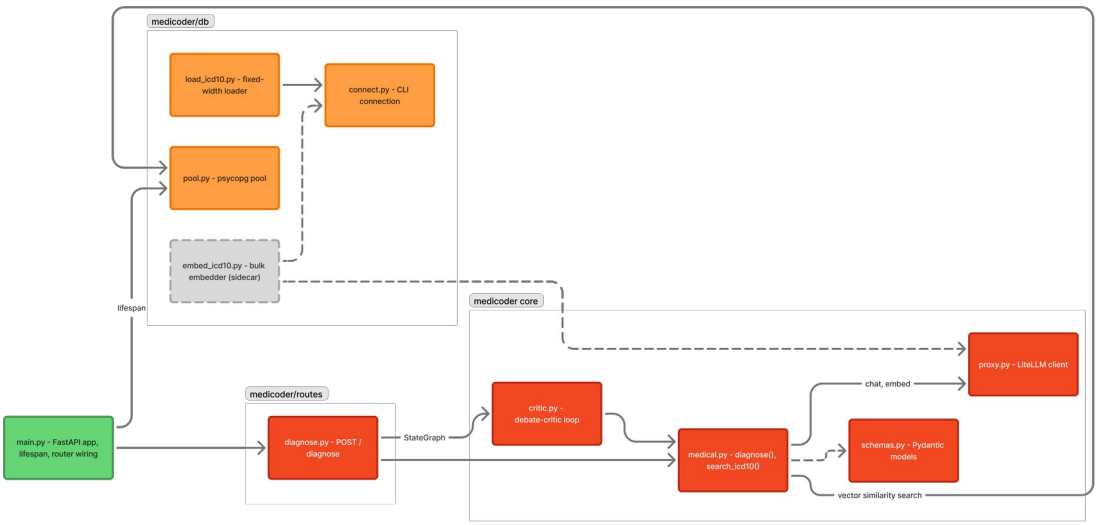


Figure 2: FastAPI Medicoder Design

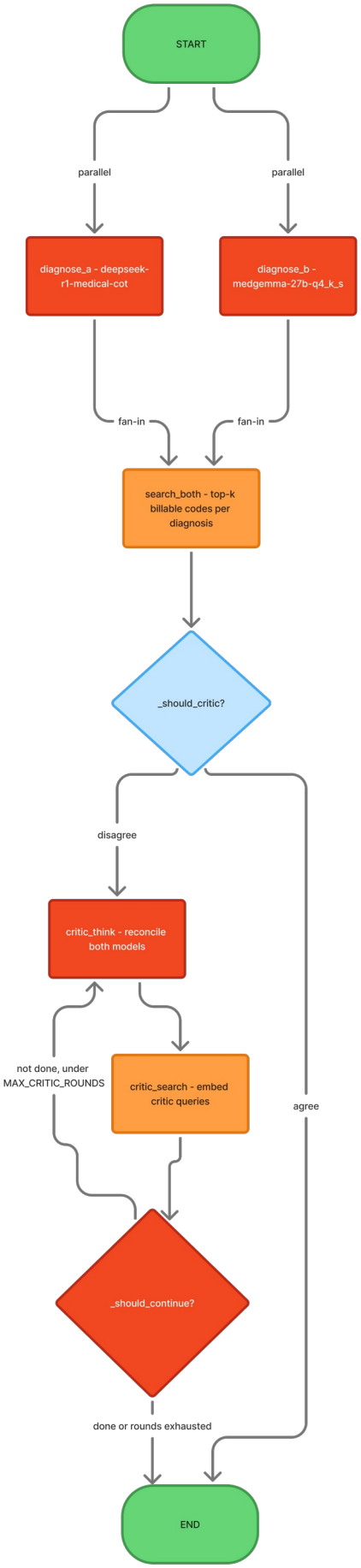


Figure 3: LangGraph structure

There are many routes used during testing, but only diagnose.py is enabled after testing was done. LangGraph was used as an orchestration tool to make sure that the LLM models chosen perform specific tasks in a specific order, as shown on the left in Figure 3. They were only allowed to generate a list of diagnoses, then search every entry on their list once to find all the applicable diagnoses. This was purposely constrained to minimise potential looping.

2 different models for inference were chosen, Deepseek-R1-Medical-COT (source: **https://huggingface.co/mradermacher/DeepSeek-R1-Medical-COT-GGUF**) and MedGemma 27B text-only (source: **https://huggingface.co/unsloth/medgemma-27b-text-it-GGUF**), as the cost of having 2 models perform inference and checking if they agreed with each other outweighed the extra cost of deploying two models. The Deepseek model was quantised to Q8_0 as this model was still relatively small (8.54 GB RAM usage estimated) at that quantisation while still providing near-F16 performance, while the MedGemma model was quantised to Q4_K_S as this quantisation put RAM usage under 16 GB. Both models ran at 0.1 temperature to minimise variability. If the models failed to produce diagnoses, they were rerun with prompts to prioritse preciseness and at a higher temperature

At the same time, a Q4_K_M quantisation of zembed-1 was used (source: **https://huggingface.co/Abiray/zembed-1-Q4_K_M-GGUF**) for the purpose of powering a local vector similarity search.

While these two models could do inference in parallel theoretically, limited available hardware required that the Ollama container unload one model before loading another for inference purposes.

If both models disagreed (in how many diagnosis codes should be assigned, or if the top-1 diagnoses identified by both models did not match by 100%), the critic loop was triggered.

The same MedGemma model used for diagnose_b was also used for the purpose of the debate-critic loop. With a maximum of 2 loops, and given more leeway to do read-only queries of the ICD-10-CM code table, it was given the diagnosis text, and the diagnoses of both models, and told to review and determine which result was correct, or to find better diagnoses.

Security/Audit Designs

Both LiteLLM and Ollama containers have no ingress ports available to outside the internal network shared by all the containers. This was done so as to minimise the attack surface if a non-authorised actor wanted to use the deployed models. Only the Medicoder FastAPI container is allowed access to LiteLLM, and only the LiteLLM and ‘sidecar’ containers have access to the Ollama container.

Parameterised queries reduce the possibility of SQL injection by the models should they end up misaligned.

As part of every call to LiteLLM, the number of prompt and output tokens, and the time taken for each request, is stored in a PostgreSQL table, for future auditing purposes.

Tradeoffs

Having the feedback of more models may have provided better code diagnosis predictions, but at increased cost, and would require more regression testing than there was time to do so, as each run took about 2 hours. One option I was not able to tune that much

Quantisation is always a trade-off between cost and memory usage, versus accuracy. Doing a regression analysis of how model performance degrades over quantisation would be a valuable data-point, but is out of scope for this project.

An attempt was made to create an Nvidia-based deployment for this project, but as this project was done on a Linux host with only access to a Radeon graphics processing unit, testing for this was not cost-feasible.

The MedGemma model is licensed under Google’s Health AI Developer Foundations Terms of Use, a less permissive software license. While this license still allows for commercial use and modifications, additional time will need to be spent auditing any software system utilising MedGemma to make sure all license requirements are complied with, as compared with more permissive licenses such as Apache License 2.0.

The current text embedding model used is a GGUF derivative of zembed-1 (link: https://huggingface.co/Abiray/zembed-1-Q4_K_M-GGUF), which is under a CC-BY-NC-4.0 license, which does not allow commercial use. However, the source model, created by Zero Entropy, is licensed under the Apache License 2.0. If this system is to be used as a baseline for future commercial development, creating a quantisation of zembed-1 will be necessary.

Assumptions

One new metric that was identified was categorical matching, rather than exact matching. The first 3 characters (letter, followed by 2 numbers, such as A38), is considered a category in ICD-10-CM nomenclature. Being able to identify the same category a patient case is assigned may be useful rather than hoping that the results being an exact match.

MedGemma was the only model tested as the critic as preliminary examinations of the DeepSeek model as the critic showed mixed performance as compared to the MedGemma as critic model. Comprehensive data collection for using the DeepSeek as critic was omitted due to the lack of time.

Results

Model Performance

Recall and F1-score (along with precision, not shown here due to its lower priority), are not measured against just the top searched result for all the diagnoses, but all results fetched between all diagnoses, weighted so as to limit the number of true positives, false positives, false negatives and true negatives to the number of diagnoses inside the dataset, or the number of diagnoses identified by the model.

How the values are calculated are as below:

TP (true positive) = Set of all fetched codes ∩ Set of all ground-truth codes
Recall = TP / number of ground-truth codes
Precision = TP / number of diagnoses
F1-score = harmonic mean of Recall and Precision

Categorical TP = Set of all categories (first 3 characters) of all fetched codes ∩ Set of all ground-truth categories
Categorical Recall = TP / number of ground-truth codes
Categorical Precision = TP / number of diagnoses
Categorical F1-score = harmonic mean of Categorical Recall and Categorical Precision

Model Performance (top 15 results returned per vector similarity search)

Model / Role	Temperature	Recall	F1	Category Recall	Category F1
DeepSeek, diagnoser	0.1	0.393	0.380	0.565	0.320
MedGemma, diagnoser	0.1	0.592	0.478	0.766	0.626
MedGemma, critic	0.1	0.581	0.498	0.735	0.637
MedGemma, critic	0.2	0.577	0.494	0.717	0.631

Critic Performance Improvement, Number of Cases each

Diagnoser Model	Critic Model	Temperature	Cases Improved	Cases Worsened	Cases Unchanged	Net Improvement
DeepSeek, diagnoser	MedGemma	0.1	26	9	15	+17
MedGemma, diagnoser	MedGemma	0.1	12	7	31	+5
Best of both models	MedGemma	0.1	10	12	28	-2
DeepSeek, diagnoser	MedGemma	0.2	29	8	11	+21
MedGemma, diagnoser	MedGemma	0.2	13	10	25	+3
Best of both models	MedGemma	0.2	12	14	22	-2

Model Efficiency (per /diagnose call)

Model	Avg Input Tokens	Avg Completion Tokens	Avg Total Tokens	Avg Number of LLM Calls
DeepSeek	508.196	607.280	1115.477	1.019
MedGemma	7446.729	1448.065	8894.794	2.682
zembed-1	92.224	N/A	N/A	4.449

Analysis of Results

The model's recall drops when critiqued, but the F1-score improves slightly as a result. Categorical recall hits a high of nearly 77% when using MedGemma by itself, but maintains performance above 70% when critiquing the result.

MedGemma is incredibly useful when critiquing DeepSeek, and still provides a net improvement when critiquing itself.

Currently, the best case, if Recall is the only priority, is to use MedGemma by itself, without a critic model. However, if F1-score is a valuable metric, utilising the current structure of having the MedGemma model critique both the output of DeepSeek and itself, is the better option.

Future Work

Separating the token counts and API calls between the diagnosers and the critics, specifically MedGemma, will be necessary for a more nuanced perspective of how much the critic costs the system, and whether the system is better off with or without the critic.

Implementing RAG with documents that advise on how to assign ICD-10-CM codes may be useful. For example, the CDC website has a document (source: **https://ftp.cdc.gov/pub/Health_Statistics/NCHS/Publications/ICD10CM/2027/ICD-10-CM-October-1-2026-FY27-Guidelines.pdf**) that provides advice on how to assign ICD-10-CM codes for a large, but not fully comprehensive, number of codes. This might provide valuable information, but a decision on how to chunk the document.

Full root-cause analysis of failures will likely be an important part of identifying how and why cases fail to be identified with the correct code. It is also entirely possible that the labels may not be 100% accurate, or ambiguous enough to allow for two different labels to be used.